

# Supervisory Control of a Two-UAV Forest Search and Rescue System with a Shared Ground Rescue Team

Chengtao Zheng A0351467Y ME6401 Topics in Mechatronics 1

**Abstract** - This report applies supervisory control theory to model a forest search and rescue system in which two unmanned aerial vehicles share one ground rescue team. For the search coverage and repair ordering problems, two specifications are designed, coverage preservation and first-faulted-first-repaired, and TCT is used to synthesise the supervisor. The resulting supervisor has 46 states and 128 transitions and disables 14 state-event pairs, all of which are controllable events. The results show that the supervisor constrains operations that violate the specifications while preserving uncontrollable behaviour.

**Index Terms** - discrete-event systems, supervisory control, controllability, TCT, search and rescue

## 1 Background

The forest environment is complex and the search area is large, so an unmanned aerial vehicle carrying an infrared camera can help ground personnel locate a trapped person. In a real mission, however, a UAV may raise a false alarm, fault or lose contact, and ground rescue resources are usually limited. Some events can be commanded, such as launch, stop and repair dispatch; events such as faults, target detection and loss of contact cannot be prevented in advance.

This report applies supervisory control theory to model two search UAVs and one shared ground rescue team. The team both answers rescue calls and recovers downed UAVs, so different tasks compete for the same resource. A supervisor is designed to coordinate the search, rescue and repair behaviour of the UAVs, so that the system still meets its operating requirements even though events such as faults and loss of contact cannot be controlled.

## 2 System description

### 2.1 Entities and states

The system contains three entities, listed in Table I. UAV1 and UAV2 are search aircraft with identical physical behaviour but different event labels. There is only one ground rescue team, and it maintains both UAVs.

TABLE I Entities

| Entity | States | Role                                                                           |
|--------|--------|--------------------------------------------------------------------------------|
| UAV1   | 6      | Search aircraft; 11 transitions                                                |
| UAV2   | 6      | Search aircraft; identical, labels offset by 12                                |
| TEAM   | 4      | The only ground rescue team: idle / rescuing / repairing UAV1 / repairing UAV2 |

The two UAVs each have six states, listed in Table II. The division into states follows the real situation. A UAV first waits in the standby state for a launch command, and once launched it enters the exploring state and searches continuously with its infrared camera. When a suspected target is found it cannot immediately be concluded that a person is in danger, so routine searching is suspended and the imagery or information is reviewed. Once the review is complete it is decided whether to call the ground team for a rescue, or to dismiss a false alarm and continue exploring. A UAV may also fault or lose contact during a search, in which case it can no longer carry out its task and must enter the down state, waiting to be recovered by the ground team rather than returning to standby by itself.

TABLE II UAV states

| State | Meaning                                                   |
|-------|-----------------------------------------------------------|
| 0     | Standby, battery sufficient (marked)                      |
| 1     | Exploring, collecting infrared imagery (marked)           |
| 2     | Suspended after a suspected target detection              |
| 3     | Down: faulted or out of contact, awaiting ground recovery |
| 4     | Review finished, awaiting a rescue decision               |
| 5     | Rescue response under way                                 |

The rescue team has four states, listed in Table III. These four states likewise follow the fact that in reality one team can carry out only one task at a time. State 0 means the team has no task and can accept a rescue call or be sent to recover a faulted UAV; state 1 means the team is carrying out a rescue and returns to idle only when it is finished. The repair states have to be split into states 2 and 3, because the team must remember which UAV it is working on: it must not be possible to be repairing UAV1 and yet have UAV2 complete its recovery. If the other UAV issues a rescue call during a repair, the model allows the team to interrupt the repair and switch to the rescue, and it does not allow the team to go and repair while a rescue is under way. Whatever the circumstances, saving life always comes first.

TABLE III Rescue team states

| State | Meaning                            |
|-------|------------------------------------|
| 0     | Idle, no duty in progress (marked) |
| 1     | Answering a rescue call            |
| 2     | Repairing UAV1                     |
| 3     | Repairing UAV2                     |

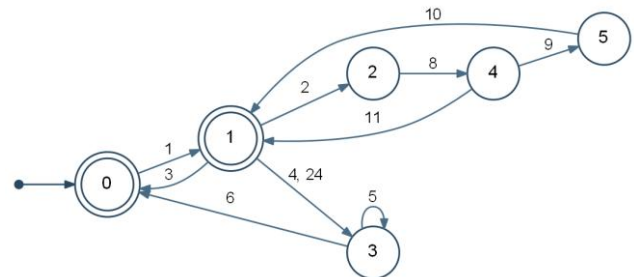


Fig. 1. UAV1 automaton. Edge labels are event numbers from Table IV. The self-loop on state 3 is the team being dispatched; UAV2 is identical with every label offset by 12.

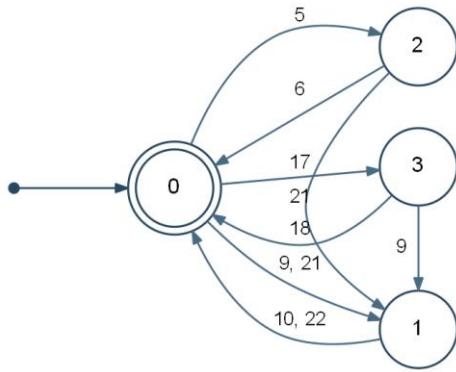


Fig. 2. Ground rescue team automaton. Transitions 2-21-1 and 3-9-1 interrupt a repair to answer a call, which is what makes rescue priority a property of the plant.

## 2.2 Events

The model has 22 events. TCT requires a controllable event to carry an odd label and an uncontrollable event an even one, so controllability can be read directly from the label. All 22 are listed in Table IV. Events 9 and 21 (the rescue calls) and 10 and 22 (the rescue completions) are shared between a UAV and the team; events 6 and 18 (recovery complete) are shared as well. Everything else belongs to a single entity.

TABLE IV Event dictionary (C = controllable, UC = uncontrollable)

| No. | Event             | C/UC | No. | Event          | C/UC |
|-----|-------------------|------|-----|----------------|------|
| 1   | launch_1          | C    | 2   | detect_1       | UC   |
| 3   | voluntary_stop_1  | C    | 4   | fault_1        | UC   |
| 5   | dispatch_repair_1 | C    | 6   | recovered_1    | UC   |
| 8   | review_complete_1 | UC   | 9   | rescue_call_1  | C    |
| 10  | rescue_complete_1 | UC   | 11  | skip_rescue_1  | C    |
| 13  | launch_2          | C    | 14  | detect_2       | UC   |
| 15  | voluntary_stop_2  | C    | 16  | fault_2        | UC   |
| 17  | dispatch_repair_2 | C    | 18  | recovered_2    | UC   |
| 20  | review_complete_2 | UC   | 21  | rescue_call_2  | C    |
| 22  | rescue_complete_2 | UC   | 23  | skip_rescue_2  | C    |
| 24  | contact_loss_1    | UC   | 26  | contact_loss_2 | UC   |

The labels are not contiguous: 7, 12, 19 and 25 are unused, because TCT requires each event to occupy a label of the same parity as its controllability, which necessarily leaves some numbers empty.

## 2.3 Specifications and structural properties

Five items constrain the behaviour of this system, and they are listed one by one below. Two of them are genuine policy choices: the plant permits the forbidden behaviour, so a supervisor has to remove it, and they are therefore expressed as specification automata. The other three are physical properties of the plant and need no specification; they are nevertheless listed here as constraints, so that the description of the system is complete.

Specification 1, coverage preservation. Whenever a controllable choice allows it, at least one UAV must remain exploring. An aircraft may not voluntarily stop while it is the only one exploring, unless it is down.

Specification 2, first faulted first repaired. If both aircraft are down, the team must be sent to the one that faulted or lost contact first.

Structural property 3, rescue priority. A rescue is never blocked by a repair, because saving life matters most.

Structural property 4, one team one duty. The team cannot rescue and repair at the same time, nor repair both aircraft at once. This is expressed by the team's four states and therefore needs no additional constraint.

Structural property 5, downed aircraft must be recovered. A faulted or out-of-contact aircraft cannot recover by itself; events 6 and 18 can occur only while the team is in the matching repair state.

TABLE V Specifications and structural properties

|   | Item                                            | Enforced by         |
|---|-------------------------------------------------|---------------------|
| 1 | Coverage preservation                           | Specification       |
| 2 | First faulted, first repaired                   | Specification       |
| 3 | A rescue is never blocked by a repair           | Structural property |
| 4 | The team performs one duty at a time            | Structural property |
| 5 | A downed aircraft must be recovered by the team | Structural property |

## 3 Modelling and results

### 3.1 Plant G

The detailed TCT modelling results and the corresponding operation steps are provided in the attachment. As obtained from TCT, the plant is the synchronous product  $G = \text{Sync}(\text{UAV1}, \text{UAV2}, \text{TEAM})$ . Events shared between entities must occur in both at once; the rest interleave freely. The result is given in Table VI.

TABLE VI The G / H / M framework

| Symbol | Object          | Obtained by                                               | States | Transitions |
|--------|-----------------|-----------------------------------------------------------|--------|-------------|
| G      | Plant           | $\text{Sync}(\text{UAV1}, \text{UAV2}, \text{TEAM})$      | 45     | 140         |
| H      | Specification   | $\text{Meet}(\text{SPEC\_COVER\_AGE}, \text{SPEC\_FIFO})$ | 20     | 234         |
| M      | Legal behaviour | $\text{Sync}(G, H)$                                       | 46     | 128         |
| M*     | Supervisor      | $\text{Supcon}(G, H)$                                     | 46     | 128         |

In Table VI, G is the plant, H the specification, M the behaviour that satisfies it, and M\* the supervisor. Two points need further explanation. First, M has 46 states while the plant has only 45, because the specification contains first-faulted-first-repaired, which needs one extra state bit to store which aircraft faulted first. Second, M and M\* have identical sizes: the legal behaviour of this design is already controllable, so Supcon removes nothing further and the supervisor coincides with M.

In addition, the three entity automata have 6, 6 and 4 states, so a full cartesian product would give 144 states, but only 45 are reachable. The reason is that the team's state is not free: if an aircraft is rescuing then the team is rescuing, and if the team is repairing UAV1 then UAV1 must be down. The synchronous product therefore genuinely couples the three entities rather than merely multiplying them, and only 45 of the 144 states survive.

The plant has no self-loops. Every event in every entity changes at least one state, so all 140 transitions are genuine moves and no transition leaves a state unchanged.

### 3.2 Specification H

A specification is expressed as an automaton whose event set is the same as the plant's. Its content is encoded by the order in which events are allowed to occur: an event that must not occur simply does not appear as an active event there. All other events are drawn as self-loops, because omitting them would itself prohibit them; and all states of the specification are marked, because the marking of the composed behaviour is determined by the plant.

For Specification 1 I use SPEC\_COVERAGE to represent coverage preservation. Its four states, 00, 10, 01 and 11, record

whether each of the two UAVs is exploring. In the figure I show the decimal equivalents, so 01 corresponds to state 1, 11 to state 3, and so on; state 2 (binary 10) means that only UAV1 is exploring and state 1 (binary 01) means that only UAV2 is exploring. Whether a voluntary stop may be allowed is then easy to judge: if only one aircraft is exploring, the specification provides no transition for that aircraft to stop voluntarily; if both are exploring, either may stop, because the other can continue the search. Faults and loss of contact also take a UAV out of exploration, so the specification updates its record after those events instead of trying to forbid them. What the supervisor can prevent is a voluntary stop; it cannot prevent a fault or a loss of contact that actually occurs.

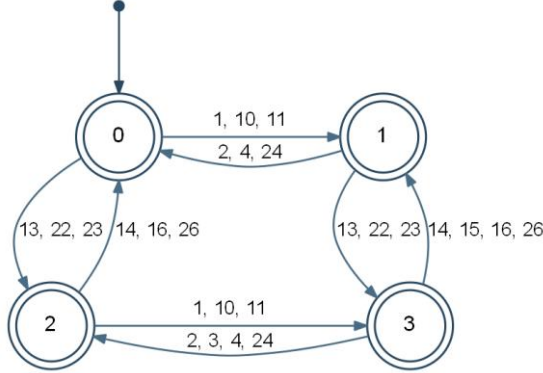


Fig. 3. SPEC\_COVERAGE. Only state-changing transitions are drawn; every other plant event is a self-loop at each state, and all four states are marked.

The second specification, SPEC\_FIFO, records the order in which the two UAVs go down. State 0 means neither is down; states 1 and 2 mean that only UAV1, or only UAV2, is down. If both fault or lose contact one after the other, the specification moves to state 3 or state 4, recording respectively that UAV1 went down first or that UAV2 went down first. This history matters, because seeing only that both are currently down does not tell you which one the team should be sent to repair first. In state 3 the event 17 that sends the team to UAV2 is not allowed; in state 4 the event 5 that sends the team to UAV1 is not allowed. In this way the supervisor restricts only a wrong dispatch order and nothing else.

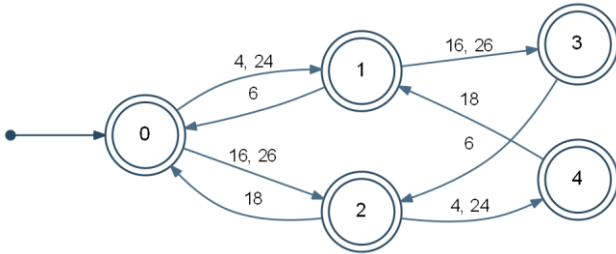


Fig. 4. SPEC\_FIFO. Event 17 is deliberately absent in state 3 and event 5 in state 4; every other plant event is a self-loop, and all five states are marked.

The two automata are combined with the meet operation into a single specification  $H = \text{Meet}(\text{SPEC\_COVERAGE}, \text{SPEC\_FIFO})$ . Table VII gives the sizes. The marked language of  $H$  is non-empty, because a mission in which nothing goes wrong satisfies both specifications.

TABLE VII Components of the specification

| Model                    | States | Transitions |
|--------------------------|--------|-------------|
| SPEC_COVERAGE            | 4      | 58          |
| SPEC_FIFO                | 5      | 90          |
| $H = \text{Meet}(\dots)$ | 20     | 234         |

### 3.3 Supervisor $M^*$

The supervisor is the supremal controllable sublanguage of the legal behaviour, computed as  $M^* = \text{Supcon}(G, H)$ . Its size is given in Table VI, its control data in Table VIII, and Fig. 5 shows how  $G$ ,  $H$  and  $M^*$  relate.

TABLE VIII Control data

| Property                        | Value                    |
|---------------------------------|--------------------------|
| Disabled state-event pairs      | 14                       |
| Event 3 (voluntary stop UAV1)   | 6 times                  |
| Event 15 (voluntary stop UAV2)  | 6 times                  |
| Event 5 (repair dispatch UAV1)  | 1 time                   |
| Event 17 (repair dispatch UAV2) | 1 time                   |
| Condat reports                  | CONTROLLABLE             |
| Removed relative to $G$         | 12 transitions, 0 states |

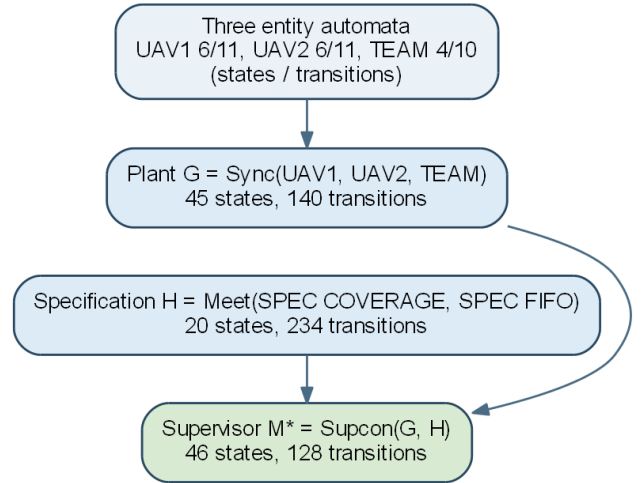


Fig. 5. Composition, specification and synthesis, with the size of each model as reported by TCT.

Two features of  $M^*$  are worth pointing out. First, it has 46 states while the plant has only 45. This has been mentioned above: the extra state is not an error but the memory that Specification 2 needs. The plant tuple  $(3, 3, 0)$ , in which both aircraft are down and the team is idle, corresponds to two different supervisor states, because the correct repair order depends on which aircraft faulted first. The supervisor therefore distinguishes two histories that the plant state alone does not.

Second, no state is removed. The plant has 45 states and the supervisor keeps all of them; the specification acts by disabling events rather than by deleting states. All 14 disabled state-event pairs involve controllable events, which is why Condat reports the supervisor as CONTROLLABLE.

### 3.4 Use of SCT

(1) Controllable and uncontrollable behaviour. The event string 1 13 3 means that UAV1 launches, UAV2 launches, and then UAV1 stops voluntarily. Because UAV2 is still carrying out the search task, this behaviour satisfies the coverage-preservation requirement. By contrast, the string 1 3 is not allowed, because UAV2 has not yet launched and stopping UAV1 at that point would leave the system without search coverage. The string 1 2 8 11, on the other hand, contains uncontrollable behaviour: UAV1 launches, a suspected target is detected, the target review is completed, and finally the rescue is skipped. Events 2 and 8 represent the target detection and the completed review, and both are uncontrollable. The supervisor cannot prevent these events from occurring; it can only restrict the subsequent controllable choices after they have occurred.

(2) Active event constraint. When UAV1 is exploring, UAV2 is in standby and the rescue team is idle, the plant's active event set is  $\Gamma_G = \{2, 3, 4, 13, 24\}$ . Events 2, 4 and 24 represent target detection, equipment fault and loss of contact, and all three are uncontrollable. Any candidate supervisor must therefore retain these three events. Suppose a candidate supervisor enabled only  $\{2, 3, 4, 13\}$  at that state, omitting event 24; that would amount to trying to prevent UAV1 from losing contact. Loss of contact is an uncontrollable event that can still occur in the plant, and the supervisor has no ability to forbid it, so that candidate state violates the active event constraint. It should be noted that the legal behaviour  $M$  obtained in this design already satisfies this constraint, so Supcon removed no states during the actual synthesis. The situation above is used to explain how the active event constraint is used to judge whether a candidate state may be kept.

(3) A controllable event disabled by the supervisor. At the same system state, the plant's active event set is  $\Gamma_G = \{2, 3, 4, 13, 24\}$ , and the active event set of the corresponding supervisor state is  $\Gamma_{M^*} = \{2, 4, 13, 24\}$ . Comparing the two sets shows that the supervisor disables only event 3, the voluntary stop of UAV1. At that moment UAV1 is the only UAV exploring, and allowing it to stop voluntarily would leave the system without search coverage, so the supervisor must disable that event. At the same time the uncontrollable events 2, 4 and 24 are retained. It follows that the supervisor restricts only the controllable choices that violate the specification and does not forbid any uncontrollable behaviour that may occur in the plant. For the case in which only UAV2 is exploring, the supervisor applies the same control logic and disables event 15, the voluntary stop of UAV2.

### 3.5 Quantitative analysis

The state and transition counts in Table VI, Table VII and Table IX come from the TCT output. Because TCT does not list the total number of events directly, this report counts the distinct event labels appearing in the transition list of each model. The sizes of the entity models, the composed plant, the specification and the final supervisor are compared in Table IX.

TABLE IX Quantitative comparison

| Model           | States | Transitions | Events |
|-----------------|--------|-------------|--------|
| UAV1            | 6      | 11          | 11     |
| UAV2            | 6      | 11          | 11     |
| TEAM            | 4      | 10          | 8      |
| Plant G         | 45     | 140         | 22     |
| SPEC_COVERAGE   | 4      | 58          | 22     |
| SPEC_FIFO       | 5      | 90          | 22     |
| Specification H | 20     | 234         | 22     |
| Supervisor M*   | 46     | 128         | 22     |

The three entities have 6, 6 and 4 states. A full cartesian product would give at most 144 combination states, but after synchronisation only 45 are reachable. This shows that the three entities are not independent. For example, when the ground team is repairing UAV1, UAV1 must be down; when a UAV is taking part in a rescue, the ground team must also be in the rescue state. These shared events and state relations rule out a large number of impossible combinations.

Combining the two specifications gives a specification H of 20 states and 234 transitions. The final supervisor M\* contains 46 states and 128 transitions. The supervisor has one state more than the plant because the first-faulted-first-repaired specification has to remember the order in which the two UAVs faulted or lost contact. When both UAVs are down and the rescue team is idle, the same plant state has

to be split into two cases in the supervisor, recording respectively that UAV1 or that UAV2 went down first.

Compared with the plant's 140 transitions, the supervisor has 128, a net reduction of 12, about 8.6 percent. Because the state sets of  $G$  and  $M^*$  are not identical, these 12 are the difference between the total transition counts and should not simply be understood as 12 edges deleted one by one on the same state graph. More directly useful control information comes from the Condat output: the supervisor disables on 14 state-event combinations in total.

Of these 14 control decisions, 12 relate to the coverage-preservation specification. When UAV1 is the only UAV carrying out the search task, the supervisor disables its voluntary stop event 3 at 6 states; when UAV2 is the only explorer, it disables event 15 at another 6 states. The remaining two control decisions come from first-faulted-first-repaired: when UAV1 goes down first, the supervisor disables event 17, the dispatch of the team to repair UAV2; when UAV2 goes down first, it disables event 5, the dispatch to repair UAV1.

All events disabled by the supervisor carry odd labels, so all of them are controllable. Uncontrollable events such as detection, fault, recovery completion and loss of contact are all retained. The supervisor therefore does not attempt to prevent any uncontrollable behaviour, and Condat accordingly reports the control data as CONTROLLABLE. All models still use the same 22 event labels, which shows that the supervisor does not remove kinds of events; it selectively disables some controllable events according to the state the system is currently in.

### 3.6 Limitations

Although I have tried my best to design this system, two limitations cannot be modelled well. First, the model has no notion of time. For example, both UAVs may be down: one went down first but is far away, and the other went down later but is nearby. If the two failures occur at about the same time, the nearby one should be repaired first, because that restores as quickly as possible a state in which at least one UAV is patrolling; but the system as designed will still choose to repair the farther one. Second, absolute rescue priority carries a starvation risk: if rescue calls keep arriving, neither specification obliges the team ever to finish a repair. Expressing a maximum waiting time for a repair would need extra states in the team automaton, but I consider this problem relatively unimportant, since rescue is always put first.

## 4 Conclusion

This report uses supervisory control theory to build a forest search and rescue model of two UAVs sharing a ground rescue team, and designs two specifications, coverage preservation and first-faulted-first-repaired. The supervisor synthesised by TCT has 46 states and 128 transitions and disables 14 state-event pairs, all of them controllable. The results show that the supervisor constrains operations that violate the specifications while preserving uncontrollable behaviour. The model currently takes no account of distance or task duration; time and position information could be added later to make it fit a real search and rescue scenario more closely.

## References

- [1] W. M. Wonham and K. Cai, Supervisory Control of Discrete-Event Systems. Cham, Switzerland: Springer, 2019.
- [2] P. C. Y. Chen, ME6401 Topics in Mechatronics 1: Supervisory Control Theory, lecture slides, National University of Singapore, 2026.
- [3] Systems Control Group, University of Toronto, TCT 2021.07.01 General Information, 2021.

## **Generative AI usage statement**

Generative AI tools were used in the preparation of this project as follows.

ChatGPT 5.6 Sol (OpenAI): I submitted paragraphs of the report that I had written and asked it to explain supervisory control theory concepts, to check whether the arguments were clear, and to improve the structure, language and paragraph organisation. The tool produced revision suggestions and some rewritten text. I checked the relevant content against my project model and the TCT output, and I selected, modified and integrated the generated text. ChatGPT was not used to replace TCT in producing the final experimental data.

DeepSeek V4.1 Flash: I supplied the modelling requirements for the UAVs, the rescue team, the events and the specifications, and asked it to help write code for model processing, synchronous composition and result tabulation. The tool produced code drafts. I checked and modified the code, and verified the state counts, transition counts and control results by running it and against the TCT output before using those results in the report.

I confirm that all AI-generated content has been checked and modified by me. I am responsible for the model design, the interpretation of the results and the final submitted content. If the course requires the complete prompt record, I will provide the relevant conversation records or screenshots separately.